COMP 4190 Assignment 4 Answer

Fidelio Ciandy, 7934456

Assignment 4, Problem 1 (Part 1)

a) $\phi(x): \mathbb{R} \to \mathbb{R}^d$

$$\text{Loss}(x,y,w) = \begin{cases} 1-2(w\cdot\phi(x))y & \text{if } (w\cdot\phi(x))y \le 0 \\ (1-(w\cdot\phi(x))y)^2 & \text{if } 0 < (w\cdot\phi(x))y \le 1 \\ 0 & \text{if } (w\cdot\phi(x))y > 1, \end{cases}$$

where $y \in \mathbb{R}$.

Compute $\nabla_w \text{Loss}(x,y,w)$

① $\nabla_w \; 1-2(w\cdot\phi(x))y$. Let $1-2(w\cdot\phi(x))y = u$

Chain Rule:

$$\frac{d}{dw} u = 1 \cdot \nabla_w u$$

$$\nabla_w \; 1-2(w\cdot\phi(x))y = -2\phi(x)y$$

② $\nabla_w (1-(w\cdot\phi(x))y)^2$. Let $1-(w\cdot\phi(x))y = u$

Chain Rule:

$$\frac{d}{dw} u^2 = 2u \cdot \nabla_w u$$

$$\nabla_w \; 1-(w\cdot\phi(x))y = -\phi(x)y$$

So, $\nabla_w (1-(w\cdot\phi(x))y)^2 = 2(1-(w\cdot\phi(x))y) \cdot -\phi(x)y$

③ $\nabla_w 0 = 0$

$$\text{So, } \nabla_w \text{Loss}(x,y,w) = \begin{cases} -2\phi(x)y \\ -2(1-(w\cdot\phi(x))y)\phi(x)y \\ 0 \end{cases}$$

b) Gradient update:

$$w^k = w^{k-1} - \eta \nabla_w \text{TrainLoss}(w^{k-1}) \text{ or,}$$

$$w \leftarrow w - \eta \nabla_w \text{TrainLoss}(w)$$

Assignment 4, Problem 1 (Part 2)

c) $d=1$, $\phi(x) = [1, x]$

$$\text{TrainLoss}(w) = \frac{1}{2}\left(\text{Loss}(x_1, y_1, w) + \text{Loss}(x_2, y_2, w)\right)$$

$w = [0, \frac{1}{2}]$,

$x_1 = -2$, $y_1 = 1$, $x_2 = -1$, $y_2 = -1$

$$\nabla_w \text{TrainLoss}(w) = \frac{1}{2}\left(\underbrace{\nabla \text{Loss}(-2, 1, [0, \tfrac{1}{2}])}_{} + \underbrace{\nabla \text{Loss}(-1, -1, [0, \tfrac{1}{2}])}_{}\right)$$

$(w \cdot \phi(x_1))y_1$

$= ([0, \frac{1}{2}] \cdot [1, -2]) \cdot 1$

$= -1 \leq 0$; case 1

$(w \cdot \phi(x_2))y_2$

$= ([0, \frac{1}{2}] \cdot [1, -1]) \cdot -1$

$= -\frac{1}{2} \cdot -1 = \frac{1}{2}$, case 2

$$\nabla_w \text{Train Loss}(w) = \frac{1}{2}\left(-2\phi(x_1)y_1 + -2\phi(x_2)y_2\left(1 - (w \cdot \phi(x_2))y_2\right)\right)$$

$$= \frac{1}{2}\left(-2[1, -2] \cdot 1 + -2 \cdot [1, -1] \cdot -1 \cdot \left(1 - \frac{1}{2}\right)\right)$$

$$= \frac{1}{2}\left([-2, 4] + [1, -1]\right) = \frac{1}{2}([-1, 3]) = \left[-\frac{1}{2}, \frac{3}{2}\right]$$

d)

$$w^1 = w^0 - \frac{1}{2}\nabla_w \text{TrainLoss}(w^0)$$

$$= [0, \tfrac{1}{2}] - \frac{1}{2} \cdot \left[-\frac{1}{2}, \frac{3}{2}\right]$$

$$= [0, \tfrac{1}{2}] - \left[-\frac{1}{4}, \frac{3}{4}\right] = \left[\frac{1}{4}, -\frac{1}{4}\right]$$

$$w^2 = w^1 - \frac{1}{2}\nabla_w \text{TrainLoss}(w^1)$$

$$= \left[\frac{1}{4}, -\frac{1}{4}\right] - \frac{1}{2}\left(\frac{1}{2}\left(\underbrace{\nabla \text{Loss}(-2, 1, [\tfrac{1}{4}, -\tfrac{1}{4}])}_{} + \underbrace{\nabla \text{Loss}(-1, 1, [\tfrac{1}{4}, -\tfrac{1}{4}])}_{}\right)\right)$$

$\left(\left[\frac{1}{4}, -\frac{1}{4}\right] \cdot [1, -2]\right) 1$

$= \frac{1}{4} + \frac{2}{4} = \frac{3}{4}$, case 2

$\left(\left[\frac{1}{4}, -\frac{1}{4}\right] \cdot [1, -1]\right) \cdot -1$

$= \left(\frac{1}{4} + \frac{1}{4}\right) \cdot -1 = -\frac{1}{2}$, case 1

$$= \left[\frac{1}{4}, -\frac{1}{4}\right] - \frac{1}{2}\left(\frac{1}{2}\left(\left(-2 \cdot [1, -2] \cdot 1 \cdot \left(1 - \frac{3}{4}\right)\right) + \left(-2[1, -1] \cdot -1\right)\right)\right)$$

$$= \left[\frac{1}{4}, -\frac{1}{4}\right] - \frac{1}{2}\left(\frac{1}{2}\left(\left[-\frac{1}{2}, 1\right] + [2, -2]\right)\right) = \left[\frac{1}{4}, -\frac{1}{4}\right] - \frac{1}{2}\left(\frac{1}{2}\left(\left[\frac{3}{2}, -1\right]\right)\right)$$

$$= \left[\frac{1}{4}, -\frac{1}{4}\right] - \frac{1}{2}\left[\frac{3}{4}, -\frac{1}{2}\right] = \left[\frac{1}{4}, -\frac{1}{4}\right] - \left[\frac{3}{8}, -\frac{1}{4}\right] = \left[-\frac{1}{8}, 0\right] //$$

signment 4, Problem 2 (Part 1)

a)

$$\text{Loss}(x, y, w) = \sigma(-(w \cdot \phi(x))y) \; ; \quad \sigma(z) = (1 + \exp(-z))^{-1}$$

Compute $\nabla_w \text{Loss}(x, y, w)$

$\nabla_w \sigma(-(w \cdot \phi(x))y)$. Let $-(w \cdot \phi(x))y = u$

Chain Rule:

$$\nabla_w \sigma(-(w \cdot \phi(x))y) = \sigma'(u) \cdot \nabla_w u$$

$$\rightarrow \quad \nabla_w u = \nabla_w -(w \cdot \phi(x))y = -\phi(x)y$$

$$\rightarrow \quad \sigma'(z) = \sigma'(1 + \exp(-z))^{-1}$$

Let $u = (1 + \exp(-z))^{-1}$

Chain Rule:

$$\sigma'(z) = \frac{d}{du} u^{-1} \cdot \frac{du}{dz} 1 + \exp(-z)$$

$$\cdot \; \frac{d}{du} u^{-1} = -u^{-2} = -(1 + \exp(-z))^{-2}$$

$$\cdot \; \frac{du}{dz} 1 + \exp(-z) = -\exp(-z)$$

$$\sigma'(z) = -(1 + \exp(-z))^{-2} \cdot \exp(-z) = \frac{\exp(-z)}{(1 + \exp(-z))^2} = \frac{\exp(-z)}{(1 + \exp(-z))} \cdot \frac{1}{(1 + \exp(-z))}$$

$$= \left(1 - \frac{1}{(1 + \exp(-z))}\right) \cdot \frac{1}{(1 + \exp(-z))}$$

$$= (1 - \sigma(z)) \cdot \sigma(z) \quad \ldots \text{ Subtitute } \sigma(z)$$

So,

$$\nabla_w \text{Loss}(x, y, w) = \sigma(-(w \cdot \phi(x))y) \cdot (1 - \sigma(-(w \cdot \phi(x))y)) \cdot -\phi(x)y$$

Assignment 4, Problem 2 (part 2)

b) $\text{Loss}(x, y, w) = \max\{1 - [(w \cdot \phi(x))y], 0\}$

$$\nabla \text{Loss}(x, y, w) = \begin{cases} 0 & , \text{ if } 1 - [(w \cdot \phi(x))y] > 0 \\ 0 & , \text{ otherwise.} \end{cases}$$

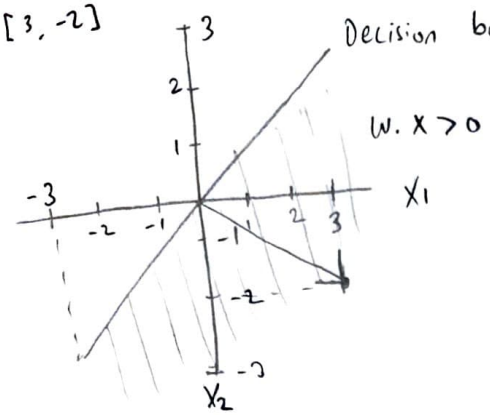
$\rightarrow \nabla_w \; 1 - [(w \cdot \phi(x))y] = 0$

- No, gradient descent is not suitable to optimize this loss function, because $w \leftarrow w - \eta \cdot 0 \cdot w = w$. Thus, w never moves, no direction is given. w never updates, so we're stuck at w^0.

Assignment 4, Problem 3

a) $W = [3, -2]$

Decision boundary : $W \cdot X = 0$



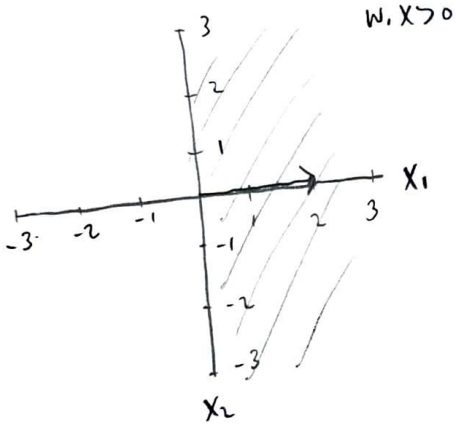
$[3, -2] \cdot [X_1, X_2] > 0$

$3X_1 - 2X_2 > 0$

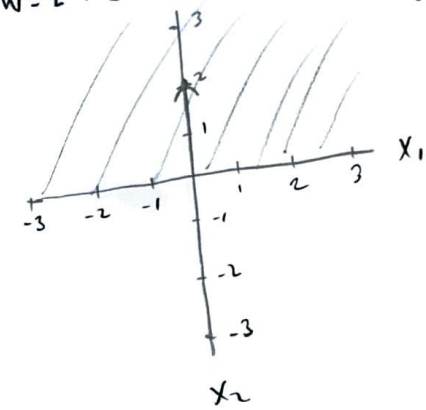
$3X_1 > 2X_2$

$X_2 < \frac{3}{2} X_1$

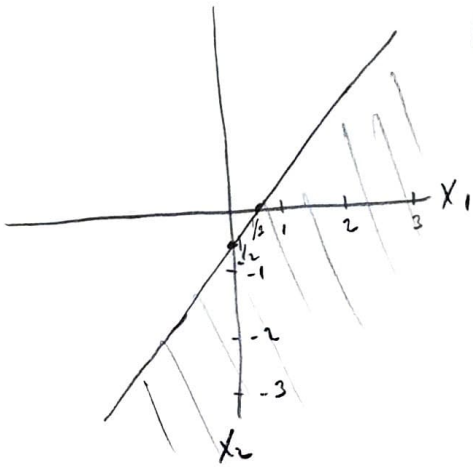
b) $w = [2, 0]$



$W = [0, 2]$



c)



$[3, -2] \cdot [X_1, X_2] \geq 1$

$3X_1 - 2X_2 \geq 1$

$3X_1 \geq 1 + 2X_2$

$X_1 \geq \frac{1 + 2X_2}{3}$

Let $X_2 = 0$,

$X_1 \geq \frac{1}{3}$

$[\frac{1}{3}, 0]$

Let $X_1 = 0$,

$0 \geq 1 + 2X_2$

$-1 \geq 2X_2$

$X_2 \leq -\frac{1}{2}$

$[0, -\frac{1}{2}]$

d) $a \leq b \iff a_i \leq b_i, \ \forall i = 1, 2, \ldots, d$

$A = \begin{bmatrix} 3 & -2 \\ 2 & 0 \end{bmatrix}, \ b = [1, 0]$

$Ax = \begin{bmatrix} 3 & -2 \\ 2 & 0 \end{bmatrix} \begin{bmatrix} X_1 \\ X_2 \end{bmatrix}$

$= \begin{bmatrix} 3X_1 - 2X_2 \\ 2X_1 + 0 \end{bmatrix}$

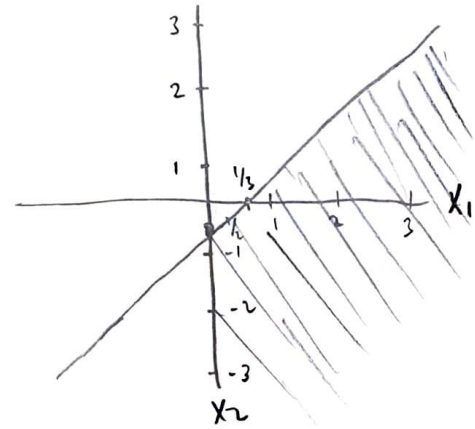
$\Rightarrow$

$Ax \geq b = [1, 0]$

$\begin{bmatrix} 3X_1 - 2X_2 \\ 2X_1 + 0 \end{bmatrix} - \begin{bmatrix} 1 \\ 0 \end{bmatrix} \geq 0$

$3X_1 - 2X_2 - 1 \geq 0$ (Part c)

$2X_1 \geq 0 \Rightarrow X_1 \geq 0$



Problem 4

- **generate_data(n=100)**
  - Generates n random 2D points sampled from a standard normal distribution ($\mu = 0$, $\sigma = 1$). Assigns label $y_i = -1$ if $x_1 + x_2 \leq 0$, and $y_i = +1$ otherwise. Returns the data matrix X and label vector y.
- **compute_loss(X, y, w, b)**
  - Computes the average logistic loss over all n data points: $\mathcal{L} = \frac{1}{n}\sum_{i=1}^{n} \log(1 + e^{-y_i(w^\top x_i + b)})$.
- **compute_gradients(X, y, w, b)**
  - Computes the gradients of the logistic loss with respect to w and b by iterating over all data points and averaging. Uses the chain rule: the derivative of $\log(1+e^u)$ with respect to u is $\frac{e^u}{1+e^u}$.
- **gradient_descent(X, y, lr=0.1, epochs=100)**
  - Initializes $w = \mathbf{0}$ and $b = 0$, then iteratively updates the parameters using gradient descent ($w \leftarrow w - \alpha\nabla_w$, $b \leftarrow b - \alpha\nabla_b$) for a fixed number of epochs. Returns the learned w and b.
- **predict(X, w, b)**
  - Produces predicted labels for each point: assigns $\hat{y}_i = +1$ if $w^\top x_i + b \geq 0$, and $\hat{y}_i = -1$ otherwise. Returns the predicted label vector $\hat{y}$.
- **plot_decision_boundary(X, y, w, b)**
  - Visualizes the dataset by plotting each point colored by its true label (blue for $+1$, red for -1). Draws the decision boundary line $w_1x_1 + w_2x_2 + b = 0$ by solving for $x_2 = -(w_1x_1 + b)/w_2$ over the range of x_1 values.

Assignment 4, Problem 4

- Compute-gradients

$$L(w,b) = \frac{1}{n} \sum_{i=1}^{n} \log\left(1 + e^{-y_i(w^T x_i + b)}\right)$$

Let $u = -y_i(w^T x_i + b)$

Chain Rule log:

$$\rightarrow \frac{d}{du} \log\left(1 + e^{-y_i(w^T x_i + b)}\right) = \frac{d}{du} \log(1+e^u) = \frac{e^u}{1+e^u}$$

$$\rightarrow \nabla_{w,b} -y_i(w^T x_i + b) \begin{cases} \nabla_w -y_i(w^T x_i + b) = -y_i x_i \\ \nabla_b -y_i(w^T x_i + b) = -y_i \end{cases}$$

$$\text{So, } \nabla_{w,b} L(w,b) = \begin{bmatrix} \frac{1}{n} \sum_{i=1}^{n} \frac{e^u}{1+e^u}(-y_i x_i) \\ \frac{1}{n} \sum_{i=1}^{n} \frac{e^u}{1+e^u}(-y_i) \end{bmatrix}$$

where $u = -y_i(w^T x_i + b)$

- gradient descent

$$w^1 = w^0 - \eta \nabla_w L(w^0, b)$$

- Plot - decision

$$w_1 x_1 + w_2 x_2 + b = 0$$

$$x_2 = \frac{-(w_1 x_1) - b}{w_2} = \frac{-(w_1 x_1 + b)}{w_2}$$

Problem 5

- **generate_data(n=200)**
  - Generates n random 2D points sampled from a standard normal distribution ($\mu = 0$, $\sigma = 1$). Assigns label $y_i = -1$ if $x_1^2 + x_2^2 \leq 1$ (inside the unit circle), and $y_i = +1$ otherwise. Returns the data matrix X and label vector y.
- **feature_map(X)**
  - Applies a polynomial feature mapping to the input data, expanding each 2D point (x_1, x_2) into a 4D feature vector $[x_1,\ x_2,\ x_1^2,\ x_2^2]$ to allow a linear classifier to capture the circular decision boundary.
- **split_data(X, y)**
  - Randomly shuffles the dataset and splits it into training (60%), validation (20%), and test (20%) sets. Returns $X_{\text{train}}, y_{\text{train}}, X_{\text{val}}, y_{\text{val}}, X_{\text{test}}, y_{\text{test}}$.
- **compute_loss(X, y, w, b)**
  - Computes the average logistic loss over all n data points (using mapped features): $\mathcal{L} = \frac{1}{n}\sum_{i=1}^{n} \log(1 + e^{-y_i(w^\top x_i + b)})$.
- **compute_gradients(X, y, w, b)**
  - Computes the average gradients of the logistic loss with respect to w and b over all data points (using mapped features). Returns ∇_w and ∇_b.
- **train(X, y, lr, epochs=200)**
  - Initializes $w = \mathbf{0}$ and $b = 0$, then runs gradient descent for a given number of epochs with learning rate `lr`. Returns the learned parameters w and b.
- **predict(X, w, b)**
  - Returns predicted labels for each point: assigns $\hat{y}_i = +1$ if $w^\top x_i + b \geq 0$, and $\hat{y}_i = -1$ otherwise.
- **compute_error(y_true, y_pred)**
  - Computes the classification error as the fraction of misclassified points: error $= \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}[\hat{y}_i \neq y_i]$.

Problem 6

- **run_mlm_example()**
  - Run masked language modeling using DistilBERT. Tokenizes the sentence `"Machine learning is very [MASK]"`, runs the model, and prints the top 10 predicted words for the masked token along with their probabilities.
- **load_data()**
  - Loads the IMDb dataset and selects a subset: 500 shuffled training samples and 200 validation samples (from the test split). Returns the train and validation splits.
- **tokenize_function(example)**
  - Tokenizes a single dataset example using the DistilBERT tokenizer with padding to `max_length=64` and truncation. Used as a mapping function over the dataset.
- **train_model(dataset)**
  - Tokenizes and formats the train and validation sets, loads `distilbert-base-uncased` for sequence classification, and fine-tunes it for one epoch using the Hugging Face `Trainer` API with fixed hyperparameters. Returns the trained `Trainer` object.